

Material de apoio ao módulo 1 - metrologia por imagem: SSH, linux e shell (bash)

Prof. Felipe Walter Dafico Pfrimer

1 Conceitos Básicos de SSH

O SSH (Secure Shell) é um protocolo de rede que permite estabelecer uma conexão segura e criptografada entre dois dispositivos, geralmente um computador local e um servidor remoto. Ele substitui protocolos inseguros como Telnet, garantindo que os dados transmitidos (como senhas e comandos) não sejam interceptados por terceiros.

O SSH utiliza criptografia para proteger a comunicação, autenticando o usuário e o servidor. Ele é amplamente usado para administração de sistemas Linux, transferência de arquivos e execução remota de comandos.

Dessa forma, o SSH pode ser utilizado para acessar máquinas remotas, como servidores e sistemas embarcados, permitindo a execução de tarefas administrativas, desenvolvimento e monitoramento de sistemas, o que é fundamental em projetos de metrologia por imagem onde o acesso remoto a equipamentos e servidores é comum.

1.1 Analogia

Imagine o SSH como enviar uma carta selada dentro de um envelope seguro e criptografado. Enquanto protocolos inseguros como o Telnet são como enviar uma carta aberta, onde qualquer pessoa no caminho pode ler o conteúdo, o SSH garante que a mensagem (seus dados e comandos) chegue lacrada e privada ao destinatário, protegida contra interceptações.

1.2 Exemplo de Acesso Remoto

Para acessar uma máquina remota via SSH, use o comando no terminal. Este comando funciona tanto em sistemas Linux (no terminal nativo) quanto no Windows 10 e 11 (no PowerShell ou CMD, desde que o OpenSSH esteja instalado — o que vem por padrão no Windows 10 versão 1803 ou superior):

Comando de Acesso Remoto via SSH

Comando	Função	Exemplo de Uso
ssh	Conectar a um servidor remoto	ssh usuario@192.168.1.100
ssh -p	Conectar em porta específica	ssh -p 2222 usuario@host
ssh -i	Conectar com chave privada	ssh -i ~/.ssh/id_rsa usuario@host
ssh-copy-id	Copiar chave pública para servidor	ssh-copy-id usuario@host

Substitua `usuario` pelo nome de usuário da máquina remota e `192.168.1.100` pelo endereço IP ou nome do host. Após executar o comando, você será solicitado a inserir a senha. Uma vez conectado, você terá acesso ao shell (interface de linha de comando) da máquina remota. O shell apresenta um prompt de comando (geralmente algo como `usuario@hostname: $`),

onde você digita os comandos para executar tarefas, navegar em diretórios, gerenciar arquivos e realizar ações administrativas como se estivesse fisicamente na máquina. O prompt será o tema da próxima seção.

2 O Prompt de Comando

O prompt de comando é a linha de texto exibida pelo shell que indica que ele está pronto para receber instruções do usuário. Geralmente, tem a forma `usuario@hostname:caminho$`, onde “usuario” é o nome do usuário logado, “hostname” é o nome da máquina, “caminho” é o diretório atual (representado por `~` para o diretório home), e o símbolo `$` indica que é um usuário comum (para root, seria `#`).

No entanto, esse formato padrão pode variar ou ser customizado pelo usuário através de variáveis de ambiente, como a `PS1` no Bash, permitindo personalizar cores, informações adicionais ou estilos conforme a preferência.

O usuário root é o superusuário do sistema, com privilégios administrativos completos. Ele tem acesso irrestrito a todos os arquivos e comandos, podendo realizar qualquer ação, incluindo aquelas que podem comprometer a segurança ou estabilidade do sistema. Por isso, é recomendado usar o usuário root apenas quando necessário e com cautela, preferindo usar um usuário comum para tarefas diárias e elevando privilégios apenas quando necessário (por exemplo, usando `sudo` no Linux).

Esse prompt serve como ponto de entrada para interagir com o sistema via linha de comando, permitindo executar programas, navegar em arquivos e configurar o ambiente de trabalho de forma eficiente e direta.

A Figura 1 ilustra um exemplo de prompt de comando no Linux, mostrando a estrutura típica com o nome do usuário (fe), o hostname (rasp), o diretório atual (`~`) e o símbolo que indica o tipo de usuário (`$`).

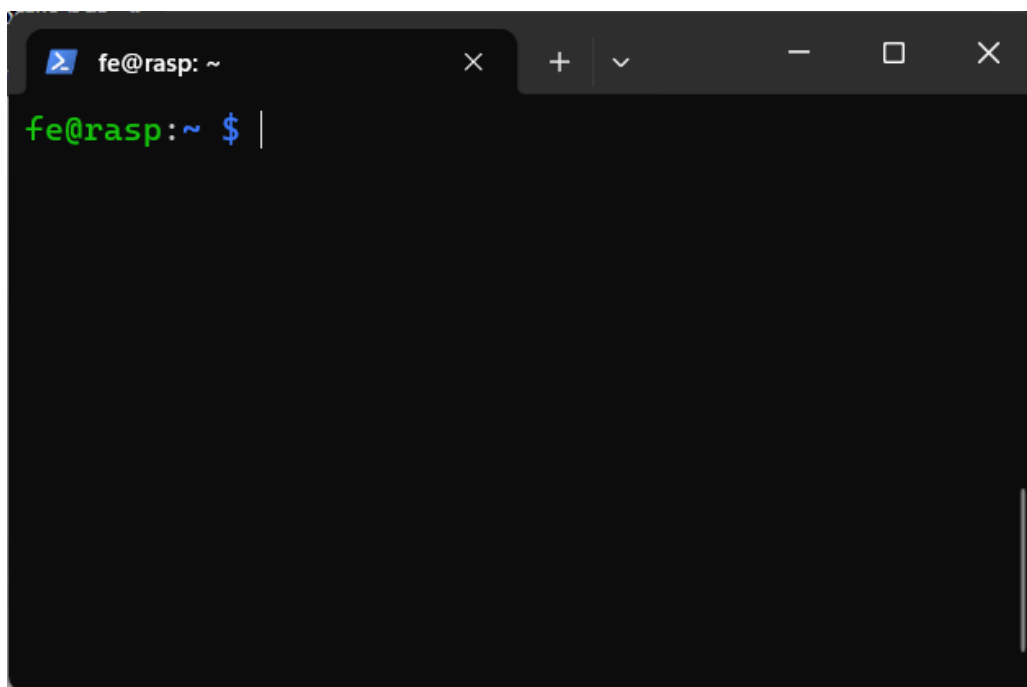


Figura 1: Exemplo de prompt de comando no Linux, exibindo `usuario@hostname: $`.

3 Fundamentos do Linux, Shell e Bash

Agora que compreendemos o prompt de comando, vamos aprofundar nos conceitos fundamentais que permitem interagir com o sistema operacional de forma eficiente. Nesta seção, veremos o que é o Linux, o que é o shell, conheceremos o Bash e aprenderemos sobre os comandos integrados que formam a base do trabalho no terminal.

3.1 O que é Linux

Linux é um sistema operacional amplamente utilizado em servidores, computadores pessoais e sistemas embarcados. Em um sistema embarcado como o Orange Pi, ele permite controlar hardware, acessar dispositivos conectados, executar programas e organizar arquivos de forma estável e automatizável.

Tecnicamente, o Linux é o *kernel*, isto é, o núcleo do sistema operacional. O kernel gerencia recursos como memória, processos, arquivos, rede e dispositivos físicos. Na prática, quando dizemos que uma máquina “roda Linux”, geralmente estamos nos referindo a uma distribuição Linux, que combina o kernel com programas, bibliotecas, ferramentas de terminal e utilitários de administração.

Essa distinção é importante porque o usuário normalmente não interage diretamente com o kernel. A interação acontece por meio de programas, como o shell, que recebe comandos digitados no terminal e solicita ao sistema operacional que execute as ações correspondentes.

Elementos Básicos de um Sistema Linux

Elemento	Função	Exemplo
kernel	Núcleo que gerencia hardware e recursos do sistema	Linux
distribuição	Conjunto formado pelo kernel e ferramentas do sistema	Ubuntu, Debian, Armbian
shell	Programa que interpreta comandos do usuário	Bash
terminal	Interface usada para digitar comandos	Console, SSH, PowerShell

Em metrologia por imagem, essa estrutura é útil porque permite conectar câmeras, salvar imagens em diretórios organizados, executar scripts de captura e automatizar tarefas repetitivas. O acesso por SSH aproveita essa base: o aluno controla remotamente um sistema embarcado, mas os comandos são executados no Linux do próprio dispositivo.

3.2 Sistema de Arquivos no Linux

O Linux organiza arquivos e dispositivos em uma estrutura hierárquica de diretórios. O diretório principal é representado por `/`, chamado de raiz. A partir dele, aparecem diretórios como `/home`, `/etc`, `/dev`, `/var` e `/tmp`.

Diretórios Comuns no Linux

Diretório	Função	Exemplo de Uso
/home	Armazena arquivos dos usuários	/home/usuario
/etc	Contém arquivos de configuração do sistema	/etc/hostname
/dev	Representa dispositivos conectados ao sistema	/dev/video0
/var	Armazena dados variáveis, como logs	/var/log
/tmp	Guarda arquivos temporários	/tmp/teste.txt

Para esta aula, o mais importante é compreender que arquivos, diretórios e alguns dispositivos seguem a mesma lógica de navegação. Por exemplo, uma webcam pode aparecer como `/dev/video0`, enquanto imagens capturadas podem ser salvas em um diretório dentro de `/home/usuario`.

3.3 O que é o Shell

O shell é um programa que atua como interface entre o usuário e o *kernel* do sistema operacional. Ele interpreta comandos digitados pelo usuário, executa programas e gerencia processos, permitindo interagir com o sistema de forma textual via linha de comando.

O termo “shell” vem do inglês e significa “casca” ou “concha”, pois o shell é como uma camada externa que envolve o kernel (o núcleo do sistema operacional), protegendo-o e facilitando a comunicação com o usuário, assim como uma casca protege o interior de uma fruta ou ostra. Essa hierarquia (usuário → shell → kernel → hardware) permite que comandos executados no shell resultem em ações diretas no hardware, como acessar uma câmera para captura de imagens, o que é essencial em aplicações de metrologia por imagem.

3.4 O que é o Bash

O Bash (Bourne Again Shell) é um dos shells mais populares e amplamente utilizados em sistemas Linux e Unix-like. Ele é uma evolução do shell Bourne (sh), oferecendo recursos avançados como histórico de comandos, autocompletar, scripting e personalização via variáveis de ambiente.

Como shell padrão em muitas distribuições Linux, o Bash facilita a interação com o sistema através de comandos de linha, sendo ideal para tarefas administrativas, automação de scripts e desenvolvimento de sistemas.

Observação: No Windows, o equivalente mais próximo ao Bash é o PowerShell, que oferece recursos similares de *scripting*, automação e linha de comando, facilitando tarefas administrativas e desenvolvimento em contextos técnicos. Alternativamente, o CMD (*Command Prompt*) é um shell mais básico no Windows, similar ao sh original, mas com funcionalidades limitadas em comparação ao Bash ou PowerShell.

3.5 Comandos Builtin do Bash

Os comandos builtin do Bash são aqueles integrados diretamente ao shell, executados sem a necessidade de iniciar um novo processo. Eles são essenciais para navegação e controle do ambiente, como `cd` (mudar diretório), `pwd` (mostrar diretório atual), `echo` (exibir texto), `.` e `source` (executar scripts no shell atual).

Para verificar se um comando é builtin, use `type comando` no terminal. Se for builtin, a saída indicará isso; caso contrário, mostrará o caminho do executável externo. Isso ajuda a entender a eficiência e o comportamento dos comandos no Bash.

4 Comandos Práticos de Navegação e Manipulação

Com a base teórica consolidada, vamos agora explorar os comandos práticos que são utilizados no dia a dia do trabalho com o terminal. Estes comandos são essenciais para organizar arquivos, configurar ambientes e preparar o sistema para tarefas de metrologia por imagem.

4.1 Comandos de Navegação

Os comandos de navegação permitem explorar e mover-se pelos diretórios do sistema de arquivos. O comando `pwd` mostra o diretório atual, `ls` lista arquivos e pastas, e `cd` muda para outro diretório.

Um símbolo importante na navegação é o `~`, que representa o diretório home do usuário. É o local padrão onde arquivos pessoais e configurações são armazenados. Por exemplo, para o usuário “fe”, o caminho seria `/home/fe`. O `~` é uma forma conveniente de se referir ao diretório home independentemente do nome do usuário, facilitando a navegação.

Exemplos práticos:

Comandos de Navegação em Diretórios

Comando	Função	Exemplo de Uso
<code>pwd</code>	Mostrar diretório atual	<code>pwd</code> → <code>/home/usuario</code>
<code>ls</code>	Listar arquivos no diretório	<code>ls /etc</code>
<code>ls -l</code>	Listar com detalhes (permissões, datas)	<code>ls -l /home</code>
<code>ls -la</code>	Listar todos incluindo ocultos	<code>ls -la ~</code>
<code>cd</code>	Mudar de diretório	<code>cd /home/usuario</code>
<code>cd ..</code>	Voltar um diretório acima	<code>cd ..</code>
<code>cd ~</code>	Ir para o diretório home	<code>cd ~</code>
<code>cd -</code>	Voltar ao diretório anterior	<code>cd -</code>

4.2 Comandos de Manipulação de Arquivos e Pastas

Os comandos de manipulação permitem criar, copiar, mover, renomear e remover arquivos e pastas. Exemplos incluem `mkdir` (criar pasta), `touch` (criar arquivo vazio), `cp` (copiar), `mv` (mover/renomear) e `rm` (remover).

Exemplos práticos com resultados esperados:

Comandos de Manipulação de Arquivos e Pastas

Comando	Função	Exemplo de Uso
mkdir	Criar diretório	mkdir nova_pasta
touch	Criar arquivo vazio	touch arquivo.txt
cp	Copiar arquivo/diretório	cp arquivo.txt copia.txt
mv	Mover ou renomear	mv arquivo.txt renomeado.txt
rm	Remover arquivo	rm copia.txt
rm -r	Remover diretório recursivo	rm -r nova_pasta
cat	Exibir conteúdo do arquivo	cat arquivo.txt

O comando `echo` é particularmente útil para exibir texto no terminal ou criar arquivos com conteúdo. Veja as principais formas de uso:

Comandos de Redirecionamento e Echo

Comando	Função	Exemplo de Uso
echo	Exibir texto no terminal	echo "Olá Mundo"
echo \$VAR	Exibir valor de variável	echo \$HOME
>	Redirecionar (sobrescrever)	echo "texto» arquivo.txt
>>	Redirecionar (anexar)	echo "mais»» arquivo.txt

4.3 Redirecionamento de Fluxo e Pipes

Além dos redirecionamentos de entrada e saída com `>` e `>>`, o Bash oferece o operador pipe (`|`), que conecta a saída de um comando à entrada de outro. Essa é uma das funcionalidades mais poderosas do shell, permitindo construir pipelines complexos para processamento de dados.

Por exemplo, é possível listar arquivos, filtrar resultados e contar linhas em uma única linha de comando. Em metrologia por imagem, pipes são úteis para encadear comandos de processamento, como converter imagens, extrair metadados e gerar estatísticas sequencialmente.

Operador Pipe (|)

Comando	Função	Exemplo de Uso
	Conectar saída de um comando à entrada de outro	ls -l grep ".txt"
grep	Filtrar linhas por padrão de texto	cat log.txt grep "erro"
sort	Ordenar linhas	ls sort
wc	Contar linhas, palavras e caracteres	ls -l wc -l
head	Exibir primeiras linhas	ls -l head -n 5
tail	Exibir últimas linhas	cat log.txt tail -n 10
less	Visualizar saída longa interativamente	cat log.txt less

4.4 Variáveis de Ambiente

As variáveis de ambiente são valores nomeados que influenciam o comportamento de programas e do próprio shell. Elas são fundamentais para configurar caminhos de execução, diretórios padrão e preferências do sistema.

No Bash, variáveis são acessadas com o prefixo `$`. A variável `PATH`, por exemplo, define os diretórios onde o shell procura por executáveis. Em projetos de metrologia, variáveis podem apontar para diretórios de imagens, configurações de câmera ou caminhos de bibliotecas de processamento.

Variáveis de Ambiente Comuns

Variável	Função	Exemplo de Uso
<code>\$HOME</code>	Diretório home do usuário	<code>echo \$HOME</code>
<code>\$PATH</code>	Caminhos de busca de executáveis	<code>echo \$PATH</code>
<code>\$USER</code>	Nome do usuário atual	<code>echo \$USER</code>
<code>\$PWD</code>	Diretório de trabalho atual	<code>echo \$PWD</code>
<code>export</code>	Definir variável de ambiente	<code>export</code> <code>CAM.DIR=/home/fe/imagens</code>

4.5 Permissões de Arquivos

O Linux utiliza um sistema de permissões para controlar acesso a arquivos e diretórios. Cada arquivo possui permissões de leitura (`r`), escrita (`w`) e execução (`x`) para três categorias: proprietário, grupo e outros.

O comando `chmod` (change mode) altera as permissões, enquanto `chown` (change owner) altera o proprietário do arquivo. Em ambientes de metrologia, é comum configurar permissões para que múltiplos usuários ou processos possam acessar imagens e dados coletados.

Gerenciamento de Permissões

Comando	Função	Exemplo de Uso
<code>ls -l</code>	Visualizar permissões	<code>ls -l arquivo.txt</code>
<code>chmod +x</code>	Adicionar permissão de execução	<code>chmod +x script.sh</code>
<code>chmod 755</code>	Definir permissões numericamente	<code>chmod 755 diretorio</code>
<code>chmod 644</code>	Arquivo: leitura para todos	<code>chmod 644 imagem.png</code>
<code>chown</code>	Alterar proprietário	<code>chown usuario arquivo.txt</code>

4.6 Processos em Segundo Plano

Em tarefas de metrologia por imagem, como capturas contínuas ou processamentos longos, é comum executar comandos em segundo plano. O Bash oferece mecanismos simples para gerenciar processos sem bloquear o terminal.

Gerenciamento de Processos

Comando	Função	Exemplo de Uso
&	Executar em segundo plano	<code>python captura.py &</code>
<code>jobs</code>	Listar processos em segundo plano	<code>jobs</code>
<code>fg</code>	Trazer processo para primeiro plano	<code>fg %1</code>
<code>Ctrl+C</code>	Interromper processo em execução	<i>(pressione no terminal)</i>
<code>nohup</code>	Executar sem interromper ao sair	<code>nohup processo.sh &</code>

4.7 Loops e Decisões em Scripts Bash

O Bash permite a criação de scripts para automatizar tarefas repetitivas e tomar decisões com base em condições. Essa funcionalidade é essencial em projetos de metrologia por imagem, onde é comum processar centenas de imagens, verificar a existência de arquivos de configuração ou executar rotinas de captura sequenciais.

Os loops `for` e `while` permitem repetir operações sobre listas de arquivos ou aguardar condições específicas. Já as estruturas condicionais `if/else` possibilitam executar comandos diferentes dependendo do resultado de testes, como verificar se um arquivo existe ou se um comando foi executado com sucesso.

Estruturas de Loop

Estrutura	Função	Exemplo de Uso
<code>for</code>	Iterar sobre uma lista	<code>for f in *.png; do echo \$f; done</code>
<code>for seq</code>	Iterar com sequência numérica	<code>for i in {1..10}; echo \$i; done</code>
<code>while</code>	Repetir enquanto condição verdadeira	<code>while [-f lock]; do sleep 1; done</code>
<code>break</code>	Sair do loop antecipadamente	<code>if [\$i -eq 5]; then break; fi</code>
<code>continue</code>	Pular para próxima iteração	<code>if [! -f \$f]; then continue; fi</code>

Estruturas Condicionais

Estrutura	Função	Exemplo de Uso
if/then	Executar se condição verdadeira	<pre>if [-f config.txt]; then echo "OK"; fi</pre>
if/else	Alternativa se falso	<pre>if [-f img.png]; then process; else echo "Falha"; fi</pre>
elif	Múltiplas condições	<pre>if [\$r -eq 0]; then ... elif [\$r -eq 1]; then ... fi</pre>
[-f]	Testar se arquivo existe	<pre>[-f /home/fe/dados.csv]</pre>
[-d]	Testar se diretório existe	<pre>[-d /home/fe/imagens]</pre>
[-s]	Testar se arquivo não está vazio	<pre>[-s log.txt]</pre>

Um exemplo prático de script que processa todas as imagens PNG em um diretório:

Exemplo de Script de Processamento

Linha	Código
1	<code>#!/bin/bash</code>
2	<code>for img in *.png; do</code>
3	<code> echo "Processando: \$img"</code>
4	<code> if [-s "\$img"]; then</code>
5	<code> python processar.py "\$img"</code>
6	<code> else</code>
7	<code> echo "Arquivo vazio: \$img"</code>
8	<code> fi</code>
9	<code>done</code>

Neste exemplo, o script itera sobre todas as imagens PNG, verifica se cada arquivo possui conteúdo (não está vazio) e, em caso positivo, executa um script Python de processamento. Caso o arquivo esteja vazio, exibe uma mensagem de aviso.

5 Escrita de Scripts Bash (Automação)

Nesta seção, vamos reunir os conceitos apresentados anteriormente para construir pequenos scripts Bash. Um script é um arquivo de texto com comandos organizados em sequência. Em vez de digitar cada comando manualmente no terminal, o usuário escreve a rotina uma vez e depois executa o arquivo sempre que precisar repetir a tarefa.

Em metrologia por imagem, essa prática é importante porque ajuda a reduzir variações introduzidas pelo operador. A mesma sequência de comandos pode ser usada para preparar diretórios, capturar imagens, registrar nomes de arquivos, processar dados e repetir medições de forma mais padronizada.

5.1 O que é um Script Bash

Um script Bash contém comandos que poderiam ser executados diretamente no terminal. A diferença é que, no script, esses comandos ficam salvos em um arquivo e são executados de forma automática pelo shell.

Isso permite transformar uma tarefa manual em uma rotina reproduzível. Por exemplo, em vez de lembrar todos os comandos necessários para capturar e organizar imagens, podemos registrar esses passos em um arquivo como `captura.sh`.

5.2 Estrutura Básica de um Script

Um script Bash normalmente começa com a linha `#!/bin/bash`, chamada de *shebang*. Ela informa ao sistema que o arquivo deve ser interpretado pelo Bash.

Script Bash Simples

Linha	Código
-------	--------

1	<code>#!/bin/bash</code>
2	<code>echo "Olá, mundo!"</code>

Depois de criar o arquivo, é necessário adicionar permissão de execução e chamá-lo pelo terminal.

Executando um Script

Comando	Função	Exemplo de Uso
<code>chmod +x</code>	Tornar o script executável	<code>chmod +x script.sh</code>
<code>./script.sh</code>	Executar o script no diretório atual	<code>./script.sh</code>

O prefixo `./` indica que o arquivo está no diretório atual. Sem esse prefixo, o shell procuraria o comando apenas nos diretórios definidos na variável `PATH`.

5.3 Uso de Parâmetros

Scripts também podem receber informações pela linha de comando. Esses valores são chamados de parâmetros ou argumentos. No Bash, `$1` representa o primeiro parâmetro, `$2` representa o segundo, e assim por diante.

Script com Parâmetro

Linha	Código
-------	--------

1	<code>#!/bin/bash</code>
2	<code>echo "Arquivo recebido: \$1"</code>

Execução com Parâmetro

Comando	Função	Exemplo de Uso
<code>./script.sh imagem.png</code>	Executar o script passando um arquivo	<code>./script.sh imagem.png</code>
<code>\$1</code>	Acessar o primeiro parâmetro dentro do script	<code>imagem.png</code>

Esse recurso é útil quando o mesmo script deve ser aplicado a arquivos diferentes, como imagens capturadas em momentos distintos.

5.4 Automação com Loops

Loops permitem repetir comandos automaticamente. Um caso comum é percorrer todos os arquivos de imagem de um diretório e executar uma ação para cada arquivo encontrado.

Processamento de Imagens com `for`

Linha	Código
1	<code>#!/bin/bash</code>
2	<code>for img in *.png; do</code>
3	<code> echo "Processando: \$img"</code>
4	<code> python processar.py "\$img"</code>
5	<code>done</code>

Neste exemplo, o Bash procura arquivos com extensão `.png`. Para cada arquivo encontrado, o nome é armazenado temporariamente na variável `img`, que pode ser usada pelos comandos dentro do loop.

5.5 Tomada de Decisão com Condicionais

Estruturas condicionais permitem executar comandos apenas quando uma condição é satisfeita. Isso é útil para evitar erros, como tentar processar um arquivo que não existe ou que está vazio.

Verificação de Arquivo com `if`

Linha	Código
1	<code>#!/bin/bash</code>
2	<code>if [-f "config.txt"]; then</code>
3	<code> echo "Arquivo encontrado"</code>
4	<code>else</code>
5	<code> echo "Arquivo não encontrado"</code>
6	<code>fi</code>

O teste `[ -f "config.txt"]` verifica a existência do arquivo. Se a condição for verdadeira, o primeiro bloco é executado; caso contrário, o bloco após `else` é executado.

5.6 Exemplo Aplicado: Captura Periódica de Imagens

Em uma rotina de aquisição, o script pode capturar imagens em intervalos regulares. O exemplo abaixo cria um nome de arquivo com data e hora, captura uma imagem e aguarda 60 segundos antes da próxima captura.

Captura Periódica de Imagens

Linha	Código
-------	--------

1	<code>#!/bin/bash</code>
2	<code>while true; do</code>
3	<code> nome="img_\$(date +%Y%m%d_%H%M%S).jpg"</code>
4	<code> echo "Capturando: \$nome"</code>
5	<code> fswebcam "\$nome"</code>
6	<code> sleep 60</code>
7	<code>done</code>

Elementos do Script de Captura

Elemento	Função	Exemplo de Uso
<code>while true</code>	Repetir continuamente	<code>while true; do ... done</code>
<code>date</code>	Gerar data e hora para o nome do arquivo	<code>date +%Y%m%d_%H%M%S</code>
<code>fswebcam</code>	Capturar imagem da webcam	<code>fswebcam imagem.jpg</code>
<code>sleep</code>	Aguardar antes da próxima repetição	<code>sleep 60</code>

Esse tipo de script ainda depende de um terminal aberto para continuar rodando. Nas próximas etapas do curso, a automação poderá ser transferida para o `systemd`, permitindo que a captura seja iniciada e controlada pelo próprio sistema.

5.7 Integração com Pipes

Scripts também podem usar pipes para combinar comandos. Isso facilita a criação de pequenos fluxos de processamento sem precisar salvar resultados intermediários em arquivos.

Uso de Pipe em Script

Comando	Função	Exemplo de Uso
<code>ls *.png</code>	Listar imagens PNG	<code>ls *.png</code>
<code>sort</code>	Ordenar a lista de arquivos	<code>ls *.png sort</code>
<code>head -n 5</code>	Exibir apenas os cinco primeiros itens	<code>ls *.png sort head -n 5</code>

No exemplo, a saída de `ls *.png` é enviada para `sort`, e a saída ordenada é enviada para `head -n 5`. Assim, cada comando executa uma parte simples da tarefa.

5.8 Considerações Finais

A escrita de scripts Bash permite transformar comandos isolados em rotinas claras, repetíveis e fáceis de registrar. Em experimentos de metrologia por imagem, isso contribui para maior organização dos dados e maior consistência na execução das etapas de aquisição e processamento.

Com os conceitos desta seção, já é possível criar scripts simples para automatizar tarefas locais. Esse conhecimento servirá de base para scripts de captura em Python e para a automação com `systemd` nas próximas aulas.

6 Conclusão

Neste módulo, vimos os conceitos e comandos básicos necessários para acessar e operar um sistema Linux remotamente. O SSH permite entrar no dispositivo pela rede; o terminal fornece a interface de trabalho; o shell interpreta os comandos; e o Bash permite tanto a execução manual quanto a criação de scripts simples.

Esses recursos formam a base para uma rotina de metrologia por imagem mais organizada e reprodutível. A partir deles, é possível navegar por diretórios, manipular arquivos, verificar permissões, usar variáveis, encadear comandos com pipes e automatizar pequenas tarefas de captura ou processamento.

Nas próximas aulas, essa base será usada para construir scripts de captura em Python e, posteriormente, automatizar sua execução com `systemd`. Assim, o processo deixa de depender de ações manuais repetidas e passa a se aproximar de um sistema de aquisição contínuo, controlado e mais confiável.